

MethylPipeline: Mathematical and Statistical Foundations

A code-backed theory reference

MethylPipeline Team

2026-03-20

Table of contents

1	Scope, Traceability, and Notation	5
1.1	Purpose	5
1.2	What Counts As “Theory” In This Repository	5
1.3	Package Map	6
1.4	Shared Data Model	6
1.5	Shared Notation	7
1.6	Cross-Cutting Assumptions	8
1.7	Reading This Book	8
2	MethylUtils	9
2.1	Scope	9
2.2	Implemented Method Inventory	9
2.3	Centroid Sufficient Statistics	10
2.4	ECDF Reconstruction	10
2.5	Two-Sample Comparison	11
2.5.1	KS Statistic on Reconstructed ECDFs	11
2.5.2	Mann-Whitney from Histogram Counts	11
2.5.3	Heterogeneity	12
2.6	Continuous Overlap	12
2.7	Multiple Testing and p-Value Aggregation	12
2.8	Canonical Biological Effect Size	13
2.9	ECDFClassifier	13
2.10	Auxiliary Beta-Based Metrics	14
3	MethylCentroid	15
3.1	Role	15
3.2	What A Centroid Stores	15
3.3	Sample-To-Centroid Update Rule	15
3.4	Coverage Capping By Binomial Thinning	16
3.5	Auto-Cap Heuristic	16
3.6	Why Histograms Matter	17
3.7	Publication Guidance	17
4	MethylDetector	18
4.1	Role	18
4.2	Alignment And Candidate Set	18
4.3	Statistical Screening	18
4.4	Biological Ranking	19
4.5	Cumulative Effect Coverage	19
4.6	Context And Chromosome Granularity	19

4.7	Exported Classifier Artifacts	20
4.8	Publication Guidance	20
5	MethylClassifier	21
5.1	Role	21
5.2	Binary Classification	21
5.3	Multi-Chromosome Binary Classification	21
5.4	One-vs-Rest (OvR) Multiclass Fusion	22
5.5	Pairwise Control Aggregation	22
5.6	Bipartite Multi-Control $\times$ Multi-Disease Aggregation	22
5.7	Calibration	23
5.8	Missing Data Semantics	23
5.9	Publication Guidance	23
6	MethylPredictor and MethylValidation	24
6.1	Role	24
6.2	MethylPredictor: Labeled Evaluation	24
6.3	Blind Prediction Summaries	25
6.4	MethylValidation: Split Strategy	25
6.5	Monte Carlo Evaluation	25
6.6	What The Validation Layer Does Not Do	26
6.7	Publication Guidance	26
7	MethylCluster	27
7.1	Role	27
7.2	Pairwise Distance Matrix	27
7.3	Library-Backed Clustering Paths	28
7.4	Centroid-Based Clustering	28
7.5	Incomplete Dirichlet-Process Branch	28
7.6	Publication Guidance	29
8	MethylMapper	30
8.1	Role	30
8.2	Interval Mapping	30
8.3	Gene-Level Combined p-Values	30
8.4	Biological Importance Score	31
8.5	Stable Gene Set Optimization	31
8.6	External Evidence Enrichment	31
8.7	Publication Guidance	32
9	MethylEnricher	33
9.1	Role	33
9.2	Over-Representation Analysis	33
9.3	Pathway Graph Construction	33
9.4	Module Enrichment Score	34
9.5	Disease Relevance Prior	34
9.6	Theme Normalization And Labels	35
9.7	Publication Guidance	35

10 MethylAlignmentQC	36
10.1 Role	36
10.2 Deterministic Metric Extraction	36
10.3 Summary Statistics	36
10.4 Log Monitoring	36
10.5 Publication Guidance	37
11 Limitations and Open Questions	38
11.1 Why This Chapter Exists	38
11.2 The “ECDF-Only Pipeline” Claim Is Too Broad	38
11.3 Dependence Is Real	38
11.4 Approximation Layers	39
11.5 Heuristic Layers	39
11.6 External Dependencies	39
11.7 Legacy Or Experimental Remnants	39
11.8 Publication-Ready Positioning	40
11.9 What Would Strengthen The Theory Further	40
Appendices	41
A MethylPipeline Theory Book	41
A.1 What This Book Covers	41
A.2 Build Requirements	41
A.2.1 HTML	41
A.2.2 PDF	41
A.3 Build Commands	42
A.4 Authoring Rules	42
A.5 Relationship To Package Docs	42

1 Scope, Traceability, and Notation

1.1 Purpose

This book documents the mathematical and statistical behavior that MethylPipeline actually implements. It is not a historical reconstruction of the first design notes, and it is not a promise about future refactors. The source of truth is the current code under `packages/`.

That rule has two consequences:

1. If the code and an older markdown document disagree, the code is canonical.
2. If the code uses an approximation or heuristic, the book says so explicitly instead of rewriting it into a cleaner but false theory story.

1.2 What Counts As “Theory” In This Repository

The repository mixes several kinds of methodology. For publication-quality documentation it is important to separate them.

Category	Meaning in this book	Typical examples in MethylPipeline
Principled	Standard statistical or numerical methods used in a recognizable form	asymptotic Kolmogorov-Smirnov testing, Storey q-values, weighted log-likelihood classification, classical clustering algorithms
Approximate	A standard method implemented through discretization, asymptotics, or reconstruction from summaries rather than raw observations	histogram-based Mann-Whitney, grid-based ECDF overlap, effective-sample-size KS
Heuristic	Engineered ranking, thresholding, or aggregation rules chosen for workflow utility rather than statistical optimality	p-values effect-size ranking, module scoring constants, pairwise control aggregation

Category	Meaning in this book	Typical examples in MethylPipeline
External-service-backed	Results depend on external databases, APIs, or stored procedures whose internal logic is not fully versioned inside this repository	Enrichr, DisGeNET, Open Targets, Azure SQL stored procedures, LLM-assisted evidence synthesis

That taxonomy is used throughout the book so the reader can tell where the pipeline is closest to a classical statistical method and where it is best interpreted as a reproducible engineering decision.

1.3 Package Map

Chapter	Primary packages	Role
Chapter 2	<code>methylutils</code>	shared mathematical layer: centroids, ECDF views, testing, effect sizes, classifier densities
Chapter 3	<code>methylcentroid</code>	centroid construction and coverage preprocessing
Chapter 4	<code>methyldetector</code>	DMP testing, filtering, and classifier export
Chapter 5	<code>methylclassifier</code>	binary, multi-chromosome, and OvR classification
Chapter 6	<code>methylpredictor</code> , <code>methylvalidation</code>	prediction metrics and Monte Carlo style validation
Chapter 7	<code>methylcluster</code>	beta-based and distance-based clustering workflows
Chapter 8	<code>methylmapper</code>	DMP-to-gene mapping and gene-level aggregation
Chapter 9	<code>methylenricher</code>	over-representation analysis, pathway graph clustering, and module scoring
Chapter 10	<code>methylalignmentqc</code>	deterministic QC parsing and summarization
Chapter 11	<code>all</code>	caveats, legacy remnants, and claims that should be softened

1.4 Shared Data Model

At the lowest level the code works with per-position methylated and unmethylated counts. For a sample s at genomic position i ,

$$x_{si} = \frac{m_{si}}{m_{si} + u_{si}},$$

where m_{si} is the methylated count and u_{si} is the unmethylated count. The code clips or guards zero-coverage positions instead of leaving them undefined.

For a centroid with N_i contributing samples at locus i , the shared sufficient statistics are

$$N_i = \sum_s 1, \quad S_{x,i} = \sum_s x_{si}, \quad S_{x^2,i} = \sum_s x_{si}^2.$$

These yield the implemented centroid mean and unbiased sample variance:

$$\hat{\mu}_i = \frac{S_{x,i}}{N_i}, \quad \hat{\sigma}_i^2 = \max\left(\frac{S_{x^2,i} - S_{x,i}^2/N_i}{\max(N_i - 1, 1)}, \varepsilon\right),$$

with a small numerical floor ε used throughout the code to avoid degenerate densities or divisions by zero.

In the ECDF-centered path, each locus also stores a binned histogram

$$\{c_{ib}\}_{b=1}^B$$

over shared methylation bin edges $0 = e_0 < e_1 < \dots < e_B = 1$. Those histograms drive both empirical distribution reconstruction and approximate rank-based testing.

1.5 Shared Notation

Symbol	Meaning
x_{si}	methylation fraction for sample s at locus i
m_{si}, u_{si}	methylated and unmethylated counts
$N_i, S_{x,i}, S_{x^2,i}$	centroid sufficient statistics
c_{ib}	count in histogram bin b for locus i
$F_i(x)$	reconstructed cumulative distribution function at locus i
$f_i(x)$	reconstructed density, usually the derivative of a monotone spline
D_i	two-sample KS statistic at locus i
U_i	Mann-Whitney U statistic at locus i
q_i	Storey-adjusted q-value
O_i	continuous overlap $\int_0^1 \min(f_{1i}, f_{2i}) dx$
w_i	feature weight used by the classifier, usually derived from detector output

1.6 Cross-Cutting Assumptions

Several assumptions recur across the repository:

- Loci are treated as conditionally independent inside the classifier score, even though methylation along the genome is correlated.
- Multiple-testing corrections are applied across many dependent loci and, downstream, across aggregated genes. This is standard practice but should not be oversold as exact finite-sample control.
- Some packages operate on summaries rather than raw per-read data. That preserves scalability, but it changes the interpretation of rank tests and distribution overlap.
- Downstream packages use external services and curated priors. Those are part of the workflow, but they are not internal statistical estimators.

1.7 Reading This Book

The book is organized from shared mathematics to downstream consumers. Readers interested in the production classifier should start with Chapter 2, then continue through Chapter 3, Chapter 4, and Chapter 5. Readers interested in biological interpretation should then continue to Chapter 8 and Chapter 9. The final chapter, Chapter 11, is intentionally blunt and should be read before any publication claim is finalized.

2 MethylUtils

2.1 Scope

`methylutils` is the mathematical spine of the repository. The core implementations live in:

- `methyl_utils/core/centroid_builder.py`
- `methyl_utils/core/distribution_views.py`
- `methyl_utils/methyl_centroid_pair.py`
- `methyl_utils/statistical_tests.py`
- `methyl_utils/ecdf_classifier.py`

Those modules define the empirical-distribution view used by centroid comparison, DMP detection, and ECDF classification. They also expose several auxiliary or legacy methods, especially beta-based distances, that are still used by downstream packages such as `methylcluster`.

2.2 Implemented Method Inventory

Area	Main code path	Status in this book
centroid sufficient statistics	<code>core/centroid_builder.py</code>	principled
ECDF reconstruction from histograms	<code>core/distribution_views.py</code>	principled numerical representation
continuous overlap integral	<code>statistical_tests.py</code>	approximate numerical integration
KS testing on reconstructed ECDFs	<code>statistical_tests.py</code>	asymptotic and grid-based
Mann-Whitney from binned counts	<code>statistical_tests.py</code>	approximate because it reconstructs rank information from histograms
Storey q-values	<code>statistical_tests.py</code>	principled, with standard tuning caveats
DerSimonian-Laird style heterogeneity	<code>statistical_tests.py</code>	principled method-of-moments adaptation
canonical biological effect size	<code>statistical_tests.py</code>	heuristic ranking score
ECDFClassifier	<code>ecdf_classifier.py</code>	principled scoring rule plus heuristic stabilizers
beta distances and overlaps	<code>metrics_core.py</code>	auxiliary; not the main detector-classifier path

2.3 Centroid Sufficient Statistics

For each locus i , the builder stores per-sample methylation fractions x_{si} together with aggregate statistics:

$$\hat{\mu}_i = \frac{S_{x,i}}{N_i}, \quad \hat{\sigma}_i^2 = \max\left(\frac{S_{x^2,i} - S_{x,i}^2/N_i}{\max(N_i - 1, 1)}, \varepsilon\right), \quad (2.1)$$

where N_i is the number of samples contributing to that locus and ε is a small numerical floor.

The centroid builder also stores count-level summaries:

$$S_{m,i} = \sum_s m_{si}, \quad S_{u,i} = \sum_s u_{si}, \quad S_{c^2,i} = \sum_s (m_{si} + u_{si})^2, \quad S_{wx^2,i} = \sum_s \frac{m_{si}^2}{m_{si} + u_{si}}, \quad (2.2)$$

which are later reused by the heterogeneity calculation. Finally, each locus accumulates histogram counts c_{ib} over shared bin edges $\{e_b\}$, producing the `binned_stats` object that drives both ECDF reconstruction and histogram-based testing.

2.4 ECDF Reconstruction

The code constructs a cumulative distribution at bin edges by normalizing cumulative bin counts:

$$\tilde{F}_i(e_b) = \frac{\sum_{r \leq b} c_{ir}}{\sum_{r=1}^B c_{ir}}, \quad b = 0, \dots, B. \quad (2.3)$$

`ECDFView` then interpolates these edge values with a monotone PCHIP spline ([Fritsch and Carlson 1980](#)). The resulting representation supplies:

- a continuous cumulative distribution $\tilde{F}_i(x)$,
- a continuous density proxy $\tilde{f}_i(x) = \max(\partial_x \tilde{F}_i(x), \varepsilon)$, and
- numerically stable means and variances via the stored sufficient statistics.

This is an empirical-distribution model, not a beta or Gaussian fit. That distinction is central to the detector-classifier path.

2.5 Two-Sample Comparison

2.5.1 KS Statistic on Reconstructed ECDFs

For aligned centroid pairs, the code evaluates the two-sample Kolmogorov-Smirnov statistic

$$D_i = \sup_{x \in [0,1]} \left| \tilde{F}_{1i}(x) - \tilde{F}_{2i}(x) \right|. \quad (2.4)$$

Implementation detail matters here: the supremum is approximated on a uniform grid rather than computed analytically, and the p -value is obtained from the large-sample Smirnov distribution ([Kolmogorov 1933](#); [Feller 1948](#)) using the harmonic mean effective sample size

$$n_{\text{eff},i} = \frac{2}{1/\max(n_{1i}, 1) + 1/\max(n_{2i}, 1)}, \quad p_i = \Pr(K \geq \sqrt{n_{\text{eff},i}} D_i). \quad (2.5)$$

This is a reasonable screening approximation, but it is not an exact finite-sample KS test on raw observations.

2.5.2 Mann-Whitney from Histogram Counts

The repository also implements a vectorized Mann-Whitney calculation directly from centroid histograms. Let $c_{ib}^{(1)}$ and $c_{ib}^{(2)}$ be the two groups' bin counts at locus i . The code reconstructs the U statistic as

$$U_i = \sum_{b=1}^B c_{ib}^{(1)} \left(\sum_{r < b} c_{ir}^{(2)} + \frac{1}{2} c_{ib}^{(2)} \right). \quad (2.6)$$

This is followed by the usual normal approximation,

$$\mu_{U,i} = \frac{n_{1i}n_{2i}}{2}, \quad Z_i = \frac{U_i - \mu_{U,i}}{\sqrt{\text{Var}(U_i)}},$$

with a tie correction derived from the combined histogram counts. The important caveat is that the code does **not** reconstruct the full rank ordering of raw methylation values; it reconstructs a rank-like summary from histogram bins. In this book that is treated as an approximation to the classical Mann-Whitney test ([Mann and Whitney 1947](#)), not as an identical procedure.

2.5.3 Heterogeneity

`dl_heterogeneity()` adapts the DerSimonian-Laird method-of-moments estimator ([DerSimonian and Laird 1986](#)) to the centroid summaries:

$$\theta_i = \frac{S_{m,i}}{S_{m,i} + S_{u,i}}, \quad Q_i = S_{wx^2,i} - \frac{S_{m,i}^2}{S_{m,i} + S_{u,i}},$$

$$\tau_i^2 = \max\left(0, \frac{Q_i - \max(N_i - 1, 0)}{\max((S_{m,i} + S_{u,i}) - S_{c^2,i}/(S_{m,i} + S_{u,i}), \varepsilon)}\right). \quad (2.7)$$

This is a useful heterogeneity diagnostic, but it should be described as a method-of-moments adaptation to stored centroid summaries, not as a full random-effects model fit.

2.6 Continuous Overlap

Once densities have been reconstructed, the code measures shared support through

$$O_i = \int_0^1 \min(\tilde{f}_{1i}(x), \tilde{f}_{2i}(x)) \, dx. \quad (2.8)$$

Numerically, the integral is evaluated on a dense grid after each density is renormalized by trapezoidal integration. This makes the overlap robust to small interpolation drift, but it remains a grid approximation.

2.7 Multiple Testing and p-Value Aggregation

For genome-wide DMP screening, `storey_qvalues()` estimates the null fraction π_0 across a grid of λ values and then computes monotone q-values ([Storey 2003](#)):

$$\hat{\pi}_0(\lambda) = \frac{\#\{p_j > \lambda\}}{(1 - \lambda)m}, \quad \hat{\pi}_0 = \min(1, \text{median}_\lambda \hat{\pi}_0(\lambda)),$$

$$q_{(i)} = \min_{j \geq i} \frac{\hat{\pi}_0 m p_{(j)}}{j}. \quad (2.9)$$

The same module also provides Fisher ([Fisher 1925](#)) and Stouffer ([Stouffer et al. 1949](#)) style p-value combination utilities. Those are not central to the detector itself, but downstream packages reuse them conceptually for gene-level aggregation.

2.8 Canonical Biological Effect Size

The repository separates statistical significance from a biological ranking score. The implemented canonical effect size is

$$\text{effect}_i = |\Delta\mu_i| (1 - O_i) \exp\left[-\lambda_{\text{var}} \left(\sqrt{\sigma_{1i}^2} + \sqrt{\sigma_{2i}^2}\right)\right]. \quad (2.10)$$

An optional mean-level weight $g(\bar{\mu}_i)$ can be applied, with the code supporting square-root, linear, saturating, and boundary-threshold variants.

This score is useful operationally, but it is not a test statistic, likelihood ratio, or posterior quantity. It is a deliberately engineered ranking rule. Publication text should therefore describe it as a heuristic biological prioritization score.

2.9 ECDFClassifier

The classifier in `ecdf_classifier.py` turns the centroid histograms into per-class densities and scores a sample $x = (x_1, \dots, x_p)$ via a weighted mean log-density:

$$\text{score}_k(x) = \frac{\sum_{i=1}^p w_i \log f_{ki}(x_i)}{\sum_{i=1}^p w_i}. \quad (2.11)$$

The class probabilities are then computed by temperature-scaled softmax:

$$P(Y = k | x) \propto \exp\left(\frac{\text{score}_k(x)}{T_{\text{eff}}}\right), \quad T_{\text{eff}} = T\sqrt{n_{\text{effective}}}, \quad (2.12)$$

where $n_{\text{effective}} = (\sum_i w_i)^2 / \sum_i w_i^2$ is the inverse-Simpson effective number of weighted loci.

Two stabilizers are important because they materially affect behavior:

1. log-densities are floored before averaging, so extremely small density values cannot dominate the score, and
2. unavailable loci are removed from the denominator by setting their effective weights to zero.

The result is a practical Naive Bayes style classifier ([Domingos and Pazzani 1997](#)), but it is not a fully calibrated generative posterior unless the independence and density assumptions actually hold. Optional Platt scaling ([Platt 1999](#)) may further recalibrate the binary probabilities after the ECDF score has been computed.

2.10 Auxiliary Beta-Based Metrics

`metrics_core.py` still contains exact beta divergences, beta overlap surrogates, and moment calculations. These are mathematically legitimate utilities, but they are **not** the main production path for DMP detection plus classification. They matter mainly because downstream clustering still uses beta-based geometry. In publication text, beta distances should therefore be described as an auxiliary branch of the repository, not as the canonical detector-classifier model.

3 MethylCentroid

3.1 Role

`methylcentroid` is the package that turns a cohort of samples into a reusable centroid object. The heavy mathematical lifting is delegated to `methylutils`, but `methylcentroid` fixes the operational semantics of what a centroid means in this repository:

- which samples are aggregated,
- how coverage outliers are handled,
- which sufficient statistics are persisted, and
- how the resulting HDF5 artifacts are organized for downstream steps.

The main implementation lives in `packages/methylcentroid/methyl_centroid/methyl_centroid.py`, with accumulation handled by `methyl_utils/core/centroid_builder.py`.

3.2 What A Centroid Stores

Each persisted centroid contains:

- genomic positions,
- context metadata,
- the sufficient statistics N_i , $S_{x,i}$, $S_{x^2,i}$,
- count summaries $S_{m,i}$, $S_{u,i}$, $S_{c^2,i}$, $S_{wx^2,i}$, and
- per-locus histogram counts used to reconstruct ECDFs.

That is why the centroid is more than a mean profile. It is a compressed empirical-distribution object designed to support both detector-side testing and classifier-side scoring.

3.3 Sample-To-Centroid Update Rule

Given one sample with per-locus methylation fraction

$$x_{si} = \frac{m_{si}}{m_{si} + u_{si}},$$

the centroid builder updates

$$N_i \leftarrow N_i + 1, \quad S_{x,i} \leftarrow S_{x,i} + x_{si}, \quad S_{x^2,i} \leftarrow S_{x^2,i} + x_{si}^2. \quad (3.1)$$

It also updates the count-derived quantities

$$\begin{aligned}
S_{m,i} &\leftarrow S_{m,i} + m_{si}, & S_{u,i} &\leftarrow S_{u,i} + u_{si}, \\
S_{c^2,i} &\leftarrow S_{c^2,i} + (m_{si} + u_{si})^2, & S_{wx^2,i} &\leftarrow S_{wx^2,i} + \frac{m_{si}^2}{m_{si} + u_{si}},
\end{aligned} \tag{3.2}$$

and increments the appropriate methylation histogram bin for x_{si} .

These updates make the centroid a compact empirical summary rather than a full sample-by-locus matrix.

3.4 Coverage Capping By Binomial Thinning

One of the most important preprocessing choices in this package is optional coverage capping. The code implements binomial thinning:

1. for any locus with coverage $n_i = m_i + u_i > n_{\text{cap}}$,
2. compute $p_i = n_{\text{cap}}/n_i$,
3. draw

$$m'_i \sim \text{Binomial}(m_i, p_i), \quad u'_i \sim \text{Binomial}(u_i, p_i). \tag{3.3}$$

This preserves the methylation fraction in expectation while reducing the influence of ultra-deep loci. In other words, it is a variance-control and robustness device, not a biological model.

The implementation exposes both manual capping and auto-capping heuristics. The thinning itself is principled; the choice of n_{cap} is still a workflow decision.

3.5 Auto-Cap Heuristic

The package documentation describes automatic coverage capping through IQR-style heuristics computed from sampled positions. That should be interpreted as an engineering rule:

$$n_{\text{cap}} \approx Q_3 + k \cdot \text{IQR},$$

with implementation-specific sampling and fallbacks. This is useful operationally, but it is not a statistically identified parameter.

3.6 Why Histograms Matter

The centroid builder stores histogram counts even though the mean and variance are already available. That is because the downstream detector and classifier do not rely only on moments. They reconstruct empirical distributions from the binned counts and then use those reconstructed distributions for overlap calculations, KS testing, and classification.

This makes `methylocentroid` a crucial bridge package: it is the step where sample-level methylation is transformed into an empirical-distribution object that later packages can reuse without reloading the full cohort.

3.7 Publication Guidance

For publication text, the safest description is:

- `methylocentroid` constructs empirical per-locus cohort summaries,
- it preserves both moments and histogram information,
- it optionally reduces extreme coverage through binomial thinning, and
- it is upstream of all ECDF-based detector and classifier logic.

The package should **not** be described as fitting a parametric methylation model. It is a summary constructor for the nonparametric empirical-distribution path.

4 MethylDetector

4.1 Role

`methyldetector` turns aligned centroids into a filtered set of differential methylation positions (DMPs) and exports the classifier artifacts used downstream. The main implementation is `packages/methyldetector/methyl_detector/core/methyldetector.py`.

Conceptually, the detector is a two-stage pipeline:

1. statistical screening of loci, and
2. biological prioritization and export.

That split is important. The code does not rely on a single monolithic significance score.

4.2 Alignment And Candidate Set

The detector starts from aligned centroid pairs, typically one control centroid and one disease centroid for a given chromosome and context. The alignment step reduces both centroids to the common set of genomic positions and propagates the empirical summaries described in [Chapter 2](#) and [Chapter 3](#).

Let $\mathcal{I}$ denote the set of aligned loci under consideration for one comparison.

4.3 Statistical Screening

The detector supports two closely related statistical paths:

- a histogram-based path through `MethylCentroidPair.compare_centroids()`, and
- a more precise ECDF path that recomputes KS statistics directly from reconstructed empirical distributions.

In both cases the active workflow is:

1. compute one p-value p_i per locus $i \in \mathcal{I}$,
2. adjust all p-values to q-values using Storey q-values,
3. keep loci satisfying the configured q-value threshold and any minimum effect thresholds.

At the theory level this is

$$\mathcal{D}_{\text{stat}} = \{i \in \mathcal{I} : q_i \leq \alpha, |\Delta\mu_i| \geq \delta_{\text{min}}\}, \quad (4.1)$$

with the exact choice of α and δ_{min} coming from configuration rather than mathematical derivation.

4.4 Biological Ranking

Once a statistically filtered set exists, the detector ranks loci using the canonical effect score introduced in Eq. 2.10. This gives a second set of priorities that is explicitly biological or practical rather than inferential.

The central design choice is that a locus can be:

- statistically strong but biologically modest,
- biologically large but not significant after multiplicity correction, or
- both.

The detector keeps these notions separate instead of compressing them into one scalar.

4.5 Cumulative Effect Coverage

One of the package’s characteristic filters is cumulative effect-mass selection. After sorting retained loci by decreasing effect size, the detector may keep only the smallest prefix whose cumulative effect mass reaches a specified fraction γ of the total:

$$\sum_{i \in S_\gamma} e_i \geq \gamma \sum_{i \in \mathcal{D}_{\text{stat}}} e_i, \quad (4.2)$$

where e_i is the per-locus effect score.

This is a useful way to compress large DMP sets, but it is a heuristic summarization rule rather than a statistical decision boundary with a formal optimality guarantee.

4.6 Context And Chromosome Granularity

The detector is designed to run at chromosome and context granularity. In practical terms this means:

- each classifier artifact is tied to a specific chromosome,
- contexts such as **CG**, **CHG**, and **CHH** can be modeled separately or together, and
- any final multiclass or multi-chromosome system is assembled later by **methylclassifier**.

That architecture keeps detection local and composable, but it also means that downstream fusion is not part of the detector’s statistical test itself.

4.7 Exported Classifier Artifacts

The detector does more than report DMPs. It also packages the information needed by the ECDF classifier:

- the filtered DMP positions,
- per-class centroid histograms over those positions,
- directions of methylation change, and
- feature weights such as effect size.

Mathematically, this means the detector defines the feature space and the feature weighting scheme for the classifier. The classifier is therefore downstream of detector choices even when its own scoring rule is fixed.

4.8 Publication Guidance

The most accurate publication description is:

- `methyldetector` performs nonparametric locus-wise screening,
- applies FDR control through Storey q-values,
- supplements significance with heuristic biological ranking,
- and exports a reduced, weighted empirical-distribution feature set.

The package should **not** be described as a pure hypothesis-testing engine if the effect-coverage filters are active, because those filters are explicitly post-test prioritization rules.

5 MethylClassifier

5.1 Role

`methylclassifier` is the orchestration layer that applies one or more binary ECDF classifiers to new samples. The mathematical core still lives in `methyl_utils/ecdf_classifier.py`, but `methylclassifier` determines how per-chromosome models, one-vs-rest bundles, and pairwise disease heads are fused into final probabilities.

The relevant implementation files are:

- `packages/methylclassifier/methyl_classifier/core/classifier.py`
- `packages/methylclassifier/methyl_classifier/core/multiclass_ovr.py`

5.2 Binary Classification

For a single binary model, the underlying score is the ECDF log-density average described in Eq. 2.11 and Eq. 2.12. From the perspective of `methylclassifier`, the main practical issues are:

- aligning the input sample to the stored DMP order,
- constructing an availability mask for missing loci, and
- optionally applying a stored Platt calibrator.

If the binary classifier is used alone, this is the cleanest statistical story in the package.

5.3 Multi-Chromosome Binary Classification

When a detector run exports one classifier per chromosome, `methylclassifier` computes chromosome-specific binary probabilities and then fuses them using normalized weights:

$$\hat{p}(y \mid x) \propto \sum_{c \in \mathcal{C}} w_c \hat{p}_c(y \mid x_c), \quad \sum_{c \in \mathcal{C}} w_c > 0. \quad (5.1)$$

The code supports several ways to determine the weights w_c :

- fixed configuration weights,
- trimmed mean effect-size weights derived from detector output,
- fitted linear, ridge, lasso, logistic, or elastic-net models trained on validation probabilities.

The last family is best understood as a second-stage meta-model that uses per-chromosome probabilities as features. It can improve prediction, but it is no longer a single coherent generative model.

5.4 One-vs-Rest (OvR) Multiclass Fusion

For multiclass problems the package builds K binary heads and fuses them into a K -class probability vector. The active implementation in `multiclass_ovr.py` uses binary probabilities or logits as intermediate scores and then applies a softmax-like normalization. In symbolic form,

$$\ell_k(x) = \log \hat{p}_k(y = k \mid x) - \log \hat{p}_k(y \neq k \mid x),$$

$$\hat{P}(Y = k \mid x) = \frac{\exp(\ell_k(x))}{\sum_{r=1}^K \exp(\ell_r(x))}. \quad (5.2)$$

This is a practical multiclass reduction. It is not guaranteed to recover a single joint Bayesian posterior unless the binary heads are themselves perfectly compatible.

5.5 Pairwise Control Aggregation

Some project layouts export only pairwise control-vs-disease detectors. In that case the package constructs the control class probability from the geometric mean of the pairwise control probabilities:

$$\hat{p}_{\text{ctrl}}(x) = \exp\left(\frac{1}{K-1} \sum_{k=1}^{K-1} \log \hat{p}_{\text{ctrl}|\text{ctrl-vs-}k}(x)\right). \quad (5.3)$$

The complement $1 - \hat{p}_{\text{ctrl}}(x)$ is then used to form a binary control-vs-rest head before final row normalization.

This is a useful aggregation rule, but it is a heuristic. The geometric mean is not derived in the code from a global multiclass likelihood.

5.6 Bipartite Multi-Control $\times$ Multi-Disease Aggregation

The package also supports bipartite layouts with several control strata and several disease strata. Here it aggregates one probability column across several pairwise experts, again using geometric means. The logic is similar to Eq. 5.3 but now applied to either the control column or the disease column depending on which class head is being built.

That construction is convenient for complex cohort hierarchies, but it should be presented as a carefully engineered reduction strategy rather than a textbook probabilistic model.

5.7 Calibration

When enabled, the package can apply Platt scaling ([Platt 1999](#)) to the raw binary outputs. In effect, one learns parameters a and b so that a raw score s is mapped to

$$\hat{p}(y = 1 \mid s) = \frac{1}{1 + \exp(as + b)}. \quad (5.4)$$

This improves probability calibration under distribution shift or class imbalance, but it should not be confused with changing the underlying classifier likelihood.

5.8 Missing Data Semantics

A practical but important detail is how the classifier handles unavailable loci. The implementation masks those loci out of the weighted average rather than imputing them with a full modeled distribution. As a result, the effective evidence for a sample depends on how many DMPs are observed for that sample.

This is a sensible engineering choice, but it means that the posterior confidence is partly a function of data availability as well as class separation.

5.9 Publication Guidance

The strongest defensible summary is:

- the binary head is an ECDF-based Naive Bayes style score with optional calibration,
- multi-chromosome fusion is a weighted ensemble,
- multiclass OvR is a reduction from binary heads to a normalized K -class score,
- pairwise control aggregation is heuristic,
- and fitted chromosome weights are second-stage discriminative models, not part of the original density model.

That framing is more accurate than calling the entire package a single Bayesian multiclass classifier.

6 MethylPredictor and MethylValidation

6.1 Role

`methylpredictor` and `methylvalidation` do not introduce a new generative methylation model. Instead they evaluate and operationalize the models exported by the detector and classifier packages.

- `methylpredictor` applies trained classifier artifacts to new cohorts and computes prediction summaries.
- `methylvalidation` creates repeated train/validation splits and aggregates run-level metrics across Monte Carlo style repetitions.

6.2 MethylPredictor: Labeled Evaluation

When labels are available, `methylpredictor` computes standard classification metrics using `sklearn.metrics`. For n labeled samples with true labels y_i and predictions $\hat{y}_i$,

$$\begin{aligned}\text{Accuracy} &= \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{\hat{y}_i = y_i\}, \\ \text{BalancedAccuracy} &= \frac{1}{K} \sum_{k=1}^K \text{Recall}_k .\end{aligned}\tag{6.1}$$

For binary classification the package also reports:

$$\text{Sensitivity} = \text{Recall}_{\text{positive}}, \quad \text{Specificity} = \text{Recall}_{\text{negative}} .$$

For multiclass settings it reports per-class precision, recall, F_1 , as well as macro and weighted F_1 summaries.

These are standard descriptive metrics. Their interpretation depends on how the evaluation cohort was assembled rather than on any special estimator inside the package.

6.3 Blind Prediction Summaries

In blind mode, `methylpredictor` cannot estimate discrimination performance because no truth labels are available. Instead it reports:

- predicted class counts,
- mean predicted probability per class, and
- mean Shannon entropy of the per-sample class probabilities.

For one sample with class probabilities $p_1, \dots, p_K$, entropy is

$$H(p) = - \sum_{k=1}^K p_k \log p_k. \quad (6.2)$$

Low entropy corresponds to confident predictions; high entropy corresponds to diffuse class assignments. This is an uncertainty summary, not an accuracy estimate.

6.4 MethylValidation: Split Strategy

`methylvalidation` implements repeated train/validation splitting by cohort. For each cohort with n samples and train fraction ρ , the code computes

$$n_{\text{train}} = \max(1, \lfloor \rho n \rfloor),$$

and then reduces n_{train} to $n - 1$ if necessary so that the validation set is non-empty for $n > 1$.

This is called “stratified” in the code because each cohort is split separately, but it is not identical to the textbook `StratifiedKfold` abstraction. The package performs independent random shuffles within each cohort and then concatenates the resulting class-specific train and validation groups.

6.5 Monte Carlo Evaluation

The validation package repeatedly:

1. generates train and validation cohorts,
2. writes run-specific project files,
3. runs the full pipeline from centroid through predictor, and
4. aggregates scalar metrics across repetitions.

If m_r denotes some metric from run r , the package reports descriptive summaries such as

$$\bar{m} = \frac{1}{R} \sum_{r=1}^R m_r, \quad s_m^2 = \frac{1}{R-1} \sum_{r=1}^R (m_r - \bar{m})^2,$$

along with empirical percentiles.

This is best interpreted as repeated internal validation of the **whole pipeline**, not as a formal confidence interval for an external deployment population.

6.6 What The Validation Layer Does Not Do

The current implementation does not attempt:

- nested hyperparameter tuning,
- exact sampling-theory confidence intervals for each metric,
- dependence-aware aggregation across repeated splits, or
- external-cohort transportability analysis.

Those omissions are not flaws in themselves, but they matter for publication wording. The package measures empirical variation across repeated pipeline runs; it does not prove generalization bounds.

6.7 Publication Guidance

For publication text:

- describe `methylnpredictor` as the evaluation and application layer for trained classifiers,
- reserve discrimination claims for labeled validation cohorts,
- treat blind-mode entropy as an uncertainty diagnostic only, and
- describe `methylnvalidation` as repeated split-sample internal validation of the end-to-end workflow.

That is both accurate to the code and defensible statistically.

7 MethylCluster

7.1 Role

`methylcluster` is an exploratory clustering package. Unlike the core detector-classifier path, it is not purely ECDF-based. It combines:

- pairwise distances built from per-locus divergences,
- standard clustering algorithms from external libraries, and
- custom centroid-based reassignment heuristics that score samples against beta distributions.

That makes it useful, but it also means it should not be described as a direct continuation of the ECDF-only statistical core.

7.2 Pairwise Distance Matrix

The first main workflow computes a sample-by-sample dissimilarity matrix. For samples a and b , the code:

1. aligns them on the intersection of available loci,
2. computes a per-locus distance using `methylutils`,
3. averages those per-locus distances.

In abstract form,

$$d(a, b) = \frac{1}{|\mathcal{I}_{ab}|} \sum_{i \in \mathcal{I}_{ab}} d_i(a, b), \quad (7.1)$$

where $\mathcal{I}_{ab}$ is the pair-specific overlap of available loci.

This is a pragmatic composite distance. Because the support set $\mathcal{I}_{ab}$ varies across pairs, the resulting matrix should be treated as an empirical dissimilarity matrix rather than assumed to satisfy all metric axioms globally.

7.3 Library-Backed Clustering Paths

Once a distance matrix exists, the package can call several standard methods:

- HDBSCAN ([Campello et al. 2015](#)) with `metric="precomputed"`,
- hierarchical agglomerative clustering with Lance-Williams style linkage ([Lance and Williams 1967](#)),
- silhouette-based selection of the number of clusters ([Rousseeuw 1987](#)),
- MDS followed by k-means ([MacQueen 1967](#)) as a fallback path.

These algorithms are principled and well-known, but the overall workflow remains only as principled as the input distance matrix.

7.4 Centroid-Based Clustering

The package also implements a custom centroid-based clustering procedure. The internal centroids are represented as `MethylCentroid` objects, but sample assignment uses beta log-densities rather than the ECDF classifier. For a sample methylation fraction x_i and cluster-specific beta parameters (α_i, β_i) ,

$$\log p(x_i | \alpha_i, \beta_i) = (\alpha_i - 1) \log x_i + (\beta_i - 1) \log(1 - x_i) - \log B(\alpha_i, \beta_i),$$

and the code averages those terms across valid loci:

$$\text{score}(x | k) = \frac{1}{|\mathcal{I}_{xk}|} \sum_{i \in \mathcal{I}_{xk}} \log p(x_i | \alpha_{ki}, \beta_{ki}). \quad (7.2)$$

This resembles a mixture-model assignment score, but the surrounding procedure is not a textbook EM algorithm. The implementation uses:

- farthest-point style initialization,
- repeated reassignment,
- cluster rescue rules for tiny clusters, and
- optional forced healthy/diseased initializations.

Those are useful engineering choices, yet they are heuristic rather than convergence-guaranteed inference.

7.5 Incomplete Dirichlet-Process Branch

The repository also contains `methy1_cluster_dp.py`, which sketches a Dirichlet-process-like clustering path. The key prior/predictive component is stubbed out, and the module is not wired into the main CLI flow. It should therefore be treated as incomplete experimental code, not as an implemented method for publication.

7.6 Publication Guidance

The most accurate description is:

- `methycluster` provides exploratory clustering utilities,
- it supports standard library algorithms over custom methylation dissimilarities,
- it also includes a beta-scored centroid reassignment procedure,
- and several parts of the workflow are heuristic or experimental.

The package should **not** be described as the natural ECDF-based unsupervised counterpart to the main classifier. The mathematical viewpoint is different.

8 MethylMapper

8.1 Role

`methylmapper` converts locus-level detector output into gene- and region-level annotations. It has two distinct mathematical layers:

1. deterministic genomic mapping through bedtools-style interval logic or an external Azure SQL stored procedure, and
2. gene-level aggregation and disease-evidence enrichment.

The relevant code lives primarily in:

- `packages/methylmapper/methyl_mapper/bedtools_mapper.py`
- `packages/methylmapper/methyl_mapper/mapper.py`
- `packages/methylmapper/methyl_mapper/gene_disease_enricher.py`

8.2 Interval Mapping

The bedtools path maps DMP coordinates to genomic features such as promoters, exons, introns, and gene bodies. This part is deterministic interval algebra rather than statistical inference. Feature-level weights may then be attached, for example promoter weights or exon weights, but those weights are biological priors chosen by the workflow rather than estimated model parameters.

The Azure SQL path calls a stored procedure (`spMapDMP2Genes`). Because the stored procedure logic is external to the repository, its exact mapping semantics should be documented as an external dependency rather than as a transparent in-repo algorithm.

8.3 Gene-Level Combined p-Values

For genes with several mapped DMPs, the package computes a signed weighted Stouffer aggregation. Given DMP p-values p_j , weights w_j , and direction signs $s_j \in \{-1, +1\}$,

$$z_j = \Phi^{-1}(1 - p_j/2), \quad \tilde{z}_j = s_j z_j,$$
$$Z_{\text{gene}} = \frac{\sum_j w_j \tilde{z}_j}{\sqrt{\sum_j w_j^2}}, \quad p_{\text{gene}} = 2 [1 - \Phi(|Z_{\text{gene}}|)]. \quad (8.1)$$

The package then applies Storey q-values to the resulting gene-level p-values.

This is a standard meta-analytic construction in form (Stouffer et al. 1949), but its interpretation requires care: mapped DMPs within one gene are correlated along the genome, so the independence assumption behind classical Stouffer aggregation is only approximate.

8.4 Biological Importance Score

`calculate_biological_importance()` combines detector statistics such as `delta_mean`, `overlap`, and `std` into a bounded gene- or region-level importance score. This is not a likelihood or hypothesis test. It is an engineered prioritization score meant to rank biologically interesting genes after mapping.

Publication text should call this what it is: a heuristic importance score that mixes detector-derived quantities and feature weights.

8.5 Stable Gene Set Optimization

The package also contains routines that choose the number of top DMPs required to obtain a stable gene set. These routines are operationally useful, but they are not estimators with standard sampling distributions. Their parameters, step sizes, and stability thresholds should therefore be documented as tuning knobs.

8.6 External Evidence Enrichment

`GeneDiseaseEnricher` queries external services such as DisGeNET (Piñero et al. 2020), Open Targets (Ochoa et al. 2023), and optional Grok-mediated evidence pipelines. This is a valuable biological annotation layer, but it is not in-repo statistical inference. The final evidence depends on:

- service availability,
- service version,
- cached responses,
- threshold profiles such as `strict` or `balanced`, and
- optional LLM-assisted synthesis.

For publication, that layer must be framed as external evidence gathering and summarization rather than as a mathematically identified part of the detector.

8.7 Publication Guidance

The most defensible description is:

- interval mapping is deterministic annotation,
- gene-level p-values are weighted Stouffer aggregates with dependence caveats,
- gene importance is heuristic,
- and disease evidence is external-service-backed annotation.

That is still scientifically useful, but it is not the same kind of object as the upstream ECDF hypothesis tests.

9 MethylEnricher

9.1 Role

`methylenricher` is a downstream interpretation layer. It takes mapped genes or weighted gene lists and organizes them into pathway-level and module-level summaries. The package mixes three kinds of methodology:

- external enrichment analysis through Enrichr ([Chen et al. 2013](#)),
- graph-based pathway grouping,
- heuristic module ranking and disease-prior labeling.

Because of that mixture, this package should be described as a discovery and prioritization layer, not as a single statistical model.

9.2 Over-Representation Analysis

The enrichment step is delegated to Enrichr through `gseapy.enrichr`. In practice this means the repository submits a gene list to an external enrichment engine and then reads back adjusted p-values, odds ratios, and gene memberships for the selected libraries.

The statistical test itself therefore lives outside the repository. The in-repo code is responsible for:

- choosing the input genes,
- selecting the enrichment libraries,
- filtering the returned tables, and
- carrying forward the adjusted p-values into later scoring steps.

Publication text should cite Enrichr ([Chen et al. 2013](#)) and record the exact library set used in the analysis.

9.3 Pathway Graph Construction

The package builds a pathway graph in which nodes are pathways and edges represent gene-set overlap. In the simplest case the edge weight is based on Jaccard similarity:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}, \quad (9.1)$$

or a closely related overlap score. Community detection is then performed with the Louvain method (Blondel et al. 2008), with connected-components fallbacks if the required package is unavailable.

This is a standard graph heuristic for organizing pathways, but the resulting modules depend materially on the chosen threshold, graph construction rule, and Louvain resolution behavior.

9.4 Module Enrichment Score

`compute_module_enrichment_score()` converts a module into a scalar ranking score by combining:

- mean $-\log_{10}(q)$ across pathways,
- number of pathways,
- number of genes,
- mean gene weight.

In normalized form the score is

$$\text{score}_{\text{enrich}} = 0.4 \tilde{m}_{-\log_{10} q} + 0.2 \tilde{n}_{\text{pathways}} + 0.2 \tilde{n}_{\text{genes}} + 0.2 \tilde{m}_{\text{gene-weight}}, \quad (9.2)$$

where each tilded quantity is clipped to $[0, 1]$ after dividing by a hand-chosen scale such as 10, 50, or 200.

This is explicitly heuristic. It is a ranking function, not an enrichment test.

9.5 Disease Relevance Prior

The package also computes a disease-relevance score by overlap with a curated gene set:

$$\text{score}_{\text{disease}} = \frac{|G_{\text{module}} \cap G_{\text{disease}}|}{|G_{\text{module}}|}. \quad (9.3)$$

When no disease set is available the package returns a neutral value of 0.5. By default the code ships with a prostate-cancer-oriented prior gene set, so that default must not be described as disease-agnostic.

The final module score is then

$$\text{score}_{\text{final}} = 0.7 \text{ score}_{\text{enrich}} + 0.3 \text{ score}_{\text{disease}}. \quad (9.4)$$

This is again a heuristic prioritization rule.

9.6 Theme Normalization And Labels

The package includes theme normalization based on substring matching against a JSON mapping file, plus discrete labels such as **High**, **Medium**, and **Low** relevance. Those labels are workflow conveniences. They should not be presented as probabilistic classifications.

9.7 Publication Guidance

The most accurate description is:

- enrichment significance comes from an external ORA engine,
- pathway modules come from graph clustering over pathway overlap,
- module ranking is heuristic,
- and disease relevance is a curated prior overlap rather than a learned posterior.

That combination can be highly useful for interpretation, but it should be framed as an annotation and prioritization pipeline rather than as formal inference.

10 MethylAlignmentQC

10.1 Role

`methylalignmentqc` is the least inferential package in the repository. It parses alignment and duplication metrics produced by external tools such as Picard and Parabricks, then rewrites them into structured JSON for downstream inspection.

The package therefore belongs in the theory book only so that readers do not over-interpret it. Its main functions are deterministic parsing, normalization, and reporting.

10.2 Deterministic Metric Extraction

The parser reads tabular metrics files, histograms, and summary blocks and maps selected fields into a fixed schema. Typical examples include:

- total reads,
- duplicate counts,
- duplication rate,
- related read-group or library statistics.

There is no new statistical estimator here beyond whatever upstream alignment tool already computed.

10.3 Summary Statistics

When the package computes summaries such as duplication rate, it is applying straightforward algebra:

$$\text{duplication rate} = \frac{\text{duplicate reads}}{\text{total reads}}. \quad (10.1)$$

These are useful QC diagnostics, but they do not form a novel methylation model.

10.4 Log Monitoring

The package also contains log-monitoring utilities that infer progress and estimated remaining time by regular-expression matching against subprocess output. Those values are operational heuristics and may change if the upstream log format changes.

10.5 Publication Guidance

In a publication, `methylationqc` should be described as:

- a QC ingestion and normalization layer,
- dependent on upstream Picard or Parabricks metrics,
- deterministic in its parsing logic, and
- not responsible for new statistical inference.

That framing keeps the package in scope without attributing more theory to it than the code actually implements.

11 Limitations and Open Questions

11.1 Why This Chapter Exists

The repository is scientifically useful, but it is not served well by a perfectly tidy story. The code mixes exact formulas, approximations, heuristics, and external-service calls. This chapter records the main caveats that should be understood before publishing claims based on the pipeline.

11.2 The “ECDF-Only Pipeline” Claim Is Too Broad

The centroid-detector-classifier path is indeed ECDF-centered. However:

- `methylcluster` uses beta-based likelihoods and distance geometry,
- `methylmapper` uses weighted Stouffer aggregation and heuristic importance scores,
- `methylenricher` uses external enrichment plus heuristic module scoring,
- `methylalignmentqc` is deterministic QC parsing, not ECDF inference.

Therefore the monorepo as a whole should not be marketed as purely ECDF-only. The safer statement is that the **core supervised detection and classification path** is ECDF-centered.

11.3 Dependence Is Real

Several methods rely on assumptions that are standard but imperfect for methylation data:

- loci within one chromosome are correlated,
- mapped DMPs within one gene are correlated,
- binary OvR heads are not guaranteed to be jointly compatible,
- repeated validation splits are not independent external studies.

This does not invalidate the workflow, but it does change how literally one should interpret p-values, q-values, and meta-analytic combinations.

11.4 Approximation Layers

The code intentionally trades exact raw-data procedures for scalable summaries:

- KS statistics are evaluated on a grid and calibrated asymptotically,
- Mann-Whitney is reconstructed from histograms rather than raw ranks,
- ECDF overlap is integrated numerically after spline reconstruction,
- some distance calculations average only over pairwise-overlapping loci.

These approximations should be reported as approximations, not hidden behind textbook method names alone.

11.5 Heuristic Layers

The following components are best described as engineering decisions:

- the canonical effect-size ranking,
- cumulative effect-mass selection,
- geometric-mean control aggregation for pairwise OvR bundles,
- module scoring constants in `methylenricher`,
- stability thresholds for choosing top DMP counts in `methylnmapper`,
- auto-cap rules for coverage trimming.

These are often useful, but their outputs are not calibrated probabilities or test statistics.

11.6 External Dependencies

Some results depend on systems whose internal logic is not fully versioned inside this repository:

- Enrichr,
- DisGeNET,
- Open Targets,
- Azure SQL stored procedures,
- optional Grok/xAI-assisted evidence synthesis.

Any publication using those layers should record service versions, access dates, and cache policies. Otherwise the results are reproducible only in a weak operational sense.

11.7 Legacy Or Experimental Remnants

Several code paths look incomplete or partially superseded:

- `packages/methylutils/comparison.py` still advertises beta-based comparison logic that does not match the active ECDF-first production path.

- `packages/methylcluster/methyl_cluster/methyl_cluster_dp.py` contains a Dirichlet-process-like branch with an unimplemented prior component.
- Some older package docs still describe a stricter or cleaner theory story than the code currently supports.

These remnants do not have to be deleted immediately, but they should not be cited as active method descriptions.

11.8 Publication-Ready Positioning

The strongest defensible manuscript framing is:

1. the core pipeline summarizes methylation into centroids with empirical distributions,
2. it detects DMPs with nonparametric, ECDF-based screening and FDR adjustment,
3. it classifies samples with weighted empirical-distribution likelihoods,
4. and it layers exploratory clustering plus biological annotation on top.

Anything stronger than that risks overstating what the code actually does.

11.9 What Would Strengthen The Theory Further

The current implementation could support stronger future papers if the repository adds:

- finite-sample benchmarking of histogram-based Mann-Whitney against raw-rank Mann-Whitney,
- calibration studies for the effective-temperature ECDF classifier,
- dependence-aware gene aggregation,
- more explicit modeling of multiclass consistency,
- and clearer retirement or isolation of experimental clustering branches.

Those are natural next steps, but they are not yet the state of the codebase.

A MethylPipeline Theory Book

This directory contains the canonical mathematical and statistical reference for MethylPipeline.

The source of truth is the code. This book is written from the implemented behavior of the packages under `packages/`, not from older design notes or aspirational API narratives. When the code and the older markdown docs disagree, the code wins.

A.1 What This Book Covers

- shared notation and data structures,
- the ECDF-centered centroid, detector, classifier, predictor, and validation stack,
- downstream packages such as clustering, mapping, enrichment, and alignment QC,
- explicit caveats for approximations, heuristics, legacy remnants, and external-service-backed steps.

A.2 Build Requirements

A.2.1 HTML

- Install [Quarto](#).

A.2.2 PDF

- Install Quarto.
- Install a TeX engine. The simplest supported path is:

```
quarto install tinytex
```

Quarto can also render with an existing TeX installation such as TeX Live or MiKTeX.

A.3 Build Commands

From the repository root on Linux or macOS:

```
quarto preview docs/theory
quarto render docs/theory --to html
quarto render docs/theory --to pdf
```

From PowerShell on Windows 11:

```
quarto preview .\docs\theory
quarto render .\docs\theory --to html
quarto render .\docs\theory --to pdf
```

To render both formats in one pass:

```
quarto render docs/theory
```

Rendered output is written to `docs/theory/_book/`.

A.4 Authoring Rules

- Treat `packages/*` code as canonical.
- Prefer primary references in `references.bib`.
- Use `[@citation-key]` for citations and `@eq-...`, `@tbl-...`, `@sec-...` for cross-references.
- Do not upgrade heuristics into theory. Label them explicitly.
- If a method depends on an external service or stored procedure, document the dependency and the reproducibility limits.

A.5 Relationship To Package Docs

The Quarto book is the canonical theory reference. Package-local `docs/THEORY.md` files should stay concise and point back to the corresponding Quarto chapter instead of becoming competing sources.

Blondel, Vincent D., Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. “Fast Unfolding of Communities in Large Networks.” *Journal of Statistical Mechanics: Theory and Experiment* 2008 (10): P10008. <https://doi.org/10.1088/1742-5468/2008/10/P10008>.

Campello, Ricardo J. G. B., Davoud Moulavi, Arthur Zimek, and Jörg Sander. 2015. “Hierarchical Density Estimates for Data Clustering, Visualization, and Outlier Detection.” *ACM Transactions on Knowledge Discovery from Data* 10 (1): 1–51. <https://doi.org/10.1145/2733381>.

- Chen, Edward Y., Christopher M. Tan, Yan Kou, et al. 2013. “Enrichr: Interactive and Collaborative HTML5 Gene List Enrichment Analysis Tool.” *BMC Bioinformatics* 14: 128. <https://doi.org/10.1186/1471-2105-14-128>.
- DerSimonian, Rebecca, and Nan Laird. 1986. “Meta-Analysis in Clinical Trials.” *Controlled Clinical Trials* 7 (3): 177–88.
- Domingos, Pedro, and Michael Pazzani. 1997. “On the Optimality of the Simple Bayesian Classifier Under Zero-One Loss.” *Machine Learning* 29 (2): 103–30.
- Feller, William. 1948. “On the Kolmogorov-Smirnov Limit Theorems for Empirical Distributions.” *The Annals of Mathematical Statistics* 19 (2): 177–89. <https://doi.org/10.1214/aoms/1177730243>.
- Fisher, Ronald A. 1925. *Statistical Methods for Research Workers*. Oliver; Boyd.
- Fritsch, F. N., and R. E. Carlson. 1980. “Monotone Piecewise Cubic Interpolation.” *SIAM Journal on Numerical Analysis* 17 (2): 238–46. <https://doi.org/10.1137/0717021>.
- Kolmogorov, A. N. 1933. “Sulla Determinazione Empirica Di Una Legge Di Distribuzione.” *Giornale Dell’Istituto Italiano Degli Attuari* 4: 83–91.
- Lance, G. N., and W. T. Williams. 1967. “A General Theory of Classificatory Sorting Strategies: 1. Hierarchical Systems.” *The Computer Journal* 9 (4): 373–80. <https://doi.org/10.1093/comjnl/9.4.373>.
- MacQueen, J. B. 1967. “Some Methods for Classification and Analysis of Multivariate Observations.” *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability* 1: 281–97.
- Mann, H. B., and D. R. Whitney. 1947. “On a Test of Whether One of Two Random Variables Is Stochastically Larger Than the Other.” *The Annals of Mathematical Statistics* 18 (1): 50–60. <https://doi.org/10.1214/aoms/1177730491>.
- Ochoa, David, Andrew Hercules, Miguel Carmona, et al. 2023. “The Next-Generation Open Targets Platform: Reimagined, Redesigned, Rebuilt.” *Nucleic Acids Research* 51 (D1): D1353–59. <https://doi.org/10.1093/nar/gkac1046>.
- Piñero, Janet, Juan M. Ramírez-Anguita, Judit Saüch-Pitarch, et al. 2020. “The DisGeNET Knowledge Platform for Disease Genomics: 2019 Update.” *Nucleic Acids Research* 48 (D1): D845–55. <https://doi.org/10.1093/nar/gkz1021>.
- Platt, John C. 1999. “Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods.” In *Advances in Large Margin Classifiers*. MIT Press.
- Rousseeuw, Peter J. 1987. “Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis.” *Journal of Computational and Applied Mathematics* 20: 53–65. [https://doi.org/10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7).

- Storey, John D. 2003. “The Positive False Discovery Rate: A Bayesian Interpretation and the q-Value.” *The Annals of Statistics* 31 (6): 2013–35. <https://doi.org/10.1214/aos/1074290335>.
- Stouffer, Samuel A., Edward A. Suchman, Leland C. DeVinney, Shirley A. Star, and Robin M. Williams. 1949. *The American Soldier: Adjustment During Army Life*. Princeton University Press.